

## 第五章 软件测试与维护 选择题解析

对应 第五章\_软件测试与维护\_选择题.md，每题附详细解析与坑点说明。

---

### 第 1 题

答案：B

解析：

这题考的是软件测试的根本目标——发现错误，而非证明无错。

测试的本质是“证伪”而非“证实”。无论测试多充分，都只能发现“存在的错误”，不能证明“不存在错误”。“0缺陷”的宣称本身就是认知错误——它只意味着“测试没发现”，不等于“真的没有”。

- A错：加长测试时间不能改变“测试无法证明无错”的本质。
- B正确：测试不能证明程序无错，只能发现错误。
- C错：自动化测试只是手段改进，同样不能证明无错。
- D错（强干扰项）：方向反了。开发者自测反而更容易有盲区（见第2题），独立测试更好。

测试 = 猎人找猎物。猎人走了一圈没看到猎物，不等于森林里没有猎物——只是没发现而已。Dijkstra名言：“测试只能证明bug的存在，不能证明bug的不存在。”

---

### 第 2 题

答案：B

解析：

这题考的是为什么需要独立测试——开发者的心理盲区。

开发者测试自己代码时，潜意识里倾向于“验证代码能跑通”（确认正确），而非“主动找错”（证伪）。这种心理倾向导致开发者容易遗漏自己埋下的错误。独立测试人员没有“作者包袱”，以找错为目的，效果更好。

- A错：技术不是核心问题，心理倾向才是。
- B正确：开发者存在心理盲区，倾向于验证而非证伪。
- C错：用例数量不是根因，心理倾向导致用例设计本身有偏差。
- D错：开发者可以写单元测试，但不应该只靠自己测。

开发者自测像“自己批改自己的作业”——潜意识里会放自己一马。独立测试像“老师批改”——以挑错为目的，更能发现问题。

---

### 第 3 题

答案：B

黑盒测试又称功能测试——把程序当黑盒，只关心输入输出（功能），不看内部结构。白盒测试又称结构测试——打开盒子看内部逻辑。

---

### 第 4 题

答案：B

解析：

这题考的是黑盒与白盒的依据差异。



黑盒测试依据需求规格说明书（外部功能描述），不需要源代码。白盒测试依据程序流程图和源代码（内部结构），没有源代码就做不了白盒。本场景只有需求规格、没有源代码，所以只能做黑盒。

- A错：白盒需要源代码，不能仅凭需求推导内部逻辑。
- B正确：黑盒依据需求规格，白盒需要源代码。
- C错：黑盒测试可以基于需求规格进行。
- D错：路径覆盖是白盒方法，需要源代码。

黑盒看“做什么”（需求），白盒看“怎么做”（源代码）。手里只有需求文档？只能黑盒。手里有源代码？可以白盒。

---

## 第 5 题

答案：C

等价类划分是黑盒方法（基于输入域划分）。语句覆盖、判定覆盖、路径覆盖都是白盒方法（基于内部结构）。

---

## 第 6 题

答案：C

判定覆盖是白盒方法（基于判定分支）。等价类划分、边界值分析、错误推测法都是黑盒方法。

---

## 第 7 题

答案：A

逻辑覆盖从弱到强：语句覆盖 < 判定覆盖 < 条件覆盖 < 判定/条件覆盖 < 条件组合覆盖 < 路径覆盖。语句覆盖最弱——只要求每条语句执行一次，连判定分支都不管。

---

## 第 8 题

答案：D

路径覆盖最强——要求所有可能路径都执行。但注意：路径覆盖最强，也不能保证发现所有错误（见第12题循环情况）。

---

## 第 9 题

答案：B

解析：

这题考的是语句覆盖与判定覆盖的包含关系——判定一定包含语句。

判定覆盖要求“每个判定的真假分支都执行一次”。真假分支都走了，意味着所有分支内的语句都执行了——所以语句自然都走到了。因此判定覆盖一定包含语句覆盖。反过来不成立：语句覆盖只要求每条语句执行一次，可能只走了一个分支就覆盖了所有语句。

团队的错误在于搞反了包含方向——以为“语句覆盖→判定覆盖”，实际是“判定覆盖→语句覆盖”。

- A错：搞反了。语句覆盖不包含判定覆盖。
- B正确：判定覆盖一定包含语句覆盖。
- C错：两者不等价（语句覆盖比判定覆盖弱）。
- D错：两者不是互不包含，是判定包含语句。

记住包含方向：判定覆盖 > 语句覆盖（判定包含语句）。判定要求真假分支都走，语句自然都走到



；但语句都走到不代表真假分支都走了。

---

## 第 10 题

答案：C

解析：

这题考的是条件覆盖与判定覆盖的不可比性——反直觉的关键点。

代码 `if (a > 0 && b > 0)`，判定只有一个，但包含两个条件a和b。

条件覆盖要求“每个条件的真假都取一次”。用例a=1 (a真)、b=-1 (b假)：条件a的真假——a=1是真，但a的假还没取到；条件b——b=-1是假。这个用例让a取了真、b取了假，但判定结果为假（只走一个分支）。

如果只用例a=1,b=1 (判定真)和a=-1,b=-1 (判定假)：判定真假都走了（满足判定覆盖），但a只取了真和假、b也取了真和假——也满足条件覆盖。

但如果用例a=1,b=-1和a=-1,b=1：条件a取了真和假、条件b取了假和真（满足条件覆盖），但判定结果都是假（不满足判定覆盖）。

所以条件覆盖满足时判定覆盖不一定满足——两者不可比。

- A错：条件覆盖不包含判定覆盖（本题就是反例）。
- B错：判定覆盖也不一定包含条件覆盖。
- C正确：两者不可比——满足条件覆盖不一定满足判定覆盖。
- D错：两者不等价。

条件覆盖vs判定覆盖是高频坑点。条件覆盖管“每个条件的真假”，判定覆盖管“每个判定的真假分支”。一个判定可能含多个条件——条件都覆盖了，判定可能只走一个分支。这就是“不可比”的原因。判断法：看判定是单条件还是多条件——单条件时两者可比，多条件时不可比。

---

## 第 11 题

答案：A

解析：

这题考的是语句覆盖与判定覆盖的实际区分——用用例集判级别。

代码 `if (x > 0) { A; }` 只有一条语句A。

• 用例集①执行x=1：x>0为真，走A分支，A执行了。只有A这一条语句，A执行了→语句覆盖达标。但判定只走了真分支，假分支没走→判定覆盖未达标。

• 用例集②执行x=1和x=-1：x=1时走真分支（A执行），x=-1时走假分支（没有语句，但假分支走了）。真假分支都走→判定覆盖达标（自然也满足语句覆盖）。

所以①达到语句覆盖，②达到判定覆盖。

• A正确：①达到语句覆盖（每条语句至少执行一次），②达到判定覆盖（真假分支都走）。语句覆盖只管“语句执行了没”，判定覆盖还管“分支都走了没”。

- B错：①没达到判定覆盖（只走真分支）。
- C错：①没达到判定覆盖。
- D错：②达到了判定覆盖，不只是语句覆盖。

语句覆盖是“每条语句都跑过”，判定覆盖是“每个判定的真假分支都跑过”。后者要求更强——不仅语句要跑，分支也要全覆盖。本题代码只有一条语句A：①走真分支覆盖A即达语句覆盖；②真假分支都走即达判定覆盖。

---

## 第 12 题

答案：A

解析：

这题考的是因果图法的识别——通过输入条件组合设计用例。

场景关键："只有当用户已登录且账户余额充足时才允许支付"——这是输入条件（登录状态、余额）和输出结果（允许/拒绝支付）之间的逻辑关系。测试人员分析这种"条件组合"来设计用例，正是因果图法的特征。

因果图法适合"输入条件之间存在逻辑关系、需要测试组合"的场景。它先用因果图分析条件与结果的逻辑（与/或/非），再转成判定表，最后生成测试用例。

• A正确：因果图法——分析输入条件（因）和输出结果（果）之间的逻辑关系，系统化设计组合测试用例。

• B错：等价类划分是按输入域划分等价类，不分析条件间的逻辑组合关系。

• C错：边界值分析是测边界值，不分析条件组合。

• D错：错误推测法是凭经验，不是系统化分析条件逻辑。

黑盒测试方法的选择：有明确输入范围→等价类划分+边界值分析；有输入条件组合逻辑（如"登录且余额足才支付"）→因果图法；凭经验补充→错误推测法。因果图法的信号词是"条件组合""逻辑关系""当...且...时"。

---

## 第 13 题

答案：B

解析：

这题考的是等价类划分的实际应用——按维度计算有效/无效等价类。

日期"YYYY-MM-DD"，按教材口径考虑四个维度：

• 有效等价类：1个（合法日期，如2023-06-15）

• 无效等价类：8个——年<1、年>9999、月<1、月>12、日<1、日>31、格式错误（如2023/06/15）、非数字（如YYYY-MM-DD字面量）

• A错：有效等价类不是4个——每个维度1个有效不等于4个有效，合法日期是一个整体。

• B正确：有效1个，无效8个。

• C错：无效不止1个。

• D错：可以计算。

有效等价类是"一个合法的完整输入"（整体1个），无效等价类按维度拆分（每个维度的越界/错误各1个）。按教材口径只算年/月/日/格式四维度，不展开闰年等复杂情况。

---

## 第 14 题

答案：B

解析：

这题考的是边界值分析是等价类的必要补充。

等价类划分取50（有效等价类的代表值），验证了"正常输入能工作"。但等价类的代表值是区间内部任意值，不测边界——而错误恰恰高发在边界（0、1、100、101）。0是无效等价类的边界，100是有效等价类的边界，都没测到。

• A错：等价类划分不是"不够多"，而是缺边界值这一步。

• B正确：缺少边界值分析，应补充0、1、100、101。

• C错：路径覆盖是白盒方法，与黑盒输入测试无关。

• D错（强干扰项）：0和100不是"无效等价类"——0是边界值（刚超出下界），100是边界值（刚在上界）。边界值分析测的是边界的"刚好"和"刚好超出"，不是增加无效等价类。

等价类划分 + 边界值分析是黄金搭档。等价类管"中间"（代表值验证正常），边界值管"两头"（边界



和刚超出)。错误高发在边界——因为编程时常犯  $>$  写成  $\geq$  这类off-by-one错误, 等价类代表值测不出来, 只有边界值能抓到。

---

## 第 15 题

答案: D

边界值分析取上点 (18、60)、离点 (17、61)、内点 (30)。100不在边界附近, 不属于边界值取值。注意: 18-60的边界是18和60, 不是0和100。

---

## 第 16 题

答案: B

解析:

这题考的是单元测试为什么以白盒为主。

单元测试的对象是单个模块, 关注模块内部的逻辑路径、分支、语句是否正确执行。这需要看源代码来设计测试用例覆盖内部路径——白盒测试。如果只用黑盒 (不看内部), 无法保证内部路径被充分覆盖。

- A错: 单元测试不能只用黑盒。
- B正确: 单元测试通常以白盒为主, 需要看源代码覆盖内部逻辑。
- C错: 不是"只用白盒", 可以结合黑盒, 但以白盒为主。
- D错: 单元测试是必需的。

单元测试=给模块做"内部体检"。模块是一颗"心脏", 黑盒只能看它"跳不跳" (功能), 白盒能看它"瓣膜开不开、心律齐不齐" (内部路径)。单元测试要深入内部, 所以以白盒为主。

---

## 第 17 题

答案: B

自顶向下集成: 上层模块先测, 下层未开发时用桩模块 (stub) 模拟下层。桩模块是被上层调用的"假"下级, 给上层返回模拟数据。口诀"自顶向下用桩"——上面先走, 下面用桩撑着。

---

## 第 18 题

答案: A

自底向上集成: 下层模块先测, 上层未开发时用驱动模块 (driver) 模拟上层。驱动模块是调用下层的"假"上级, 给下层发测试数据。口诀"自底向上用驱动"——下面先走, 上面用驱动推着。

---

## 第 19 题

答案: B

解析:

这题考的是 $\alpha$ 测试与 $\beta$ 测试的精确定义——主体和环境 (以张海藩教材为准)。

- $\alpha$ 测试: 用户在开发环境中测试 (主体是用户, 不是开发者)
- $\beta$ 测试: 用户在实际使用环境中测试 (主体是用户, 环境是用户方)

公司的安排有两个错误: ①让开发者做 $\alpha$ 测试 (主体错了—— $\alpha$ 是用户做); ② $\beta$ 测试在公司做 (环境错了—— $\beta$ 在用户实际环境)。





- A错（强干扰项）： $\beta$ 测试的环境确实应在用户方，但搞错的更关键是 $\alpha$ 测试的主体—— $\alpha$ 测试不该由开发者做。
  - B正确： $\alpha$ 测试是用户在开发环境测（不是开发者）， $\beta$ 测试是用户在实际使用环境测。
  - C错： $\beta$ 测试是用户做，不是开发者。
  - D错：不只是地点搞反，主体也搞错了（ $\alpha$ 不该开发者做）。
- $\alpha$ 和 $\beta$ 的主体都是用户，区别在环境： $\alpha$ 在开发方环境（用户来开发方测）， $\beta$ 在用户方环境（用户在自己家测）。最容易记错的是 $\alpha$ 的主体—— $\alpha$ 不是开发者测，是用户来开发方测。以张海藩教材为准，部分教材口径略有差异。
- 

## 第 20 题

答案：B

解析：

这题考的是回归测试的作用——防止修改引入新错误。

修了一个bug后直接上线，另一个功能坏了——这说明修改产生了副作用（改A影响B）。回归测试就是在修改后重新运行之前的测试用例，确保原有功能没被破坏。

- A错：代码质量不是根因，缺回归测试流程才是。
- B正确：缺少回归测试——修改后应重新运行之前用例。
- C错：重写系统是过度反应。
- D错：不是测试人员失职，是流程缺了回归测试这一步。

修改代码像"动手术"——手术成功了（bug修了），但可能伤到旁边器官（其他功能）。回归测试就是术后全身检查——不只查手术部位，全身都要查一遍，确保没有连带损伤。

---

## 第 21 题

答案：C

完善性维护占比最大（约50-66%）——用户用着用着提出新需求、要求增强功能，是维护的主要来源。改正性维护（修bug）约17-21%，适应性维护（适应环境变化）约18-25%，预防性维护最少。口诀"完善最多"。

---

## 第 22 题

答案：A

解析：

这题考的是三种维护类型的区分——以及"交付前"的时间陷阱。

三条工单：

- ①C/S改B/S：适应环境变化（新平台）→适应性维护
- ②利息计算错误：修复已上线系统的bug→改正性维护
- ③增加导出PDF：用户要求新功能→完善性维护

陷阱：交付前测试阶段发现的bug。维护的定义是"交付后的活动"——交付前发现的bug属于开发阶段，不构成"维护"，属于开发过程中的正常修错。

- A正确：①适应性 ②改正性 ③完善性；交付前bug不算维护。
- B、C、D错：都把交付前的bug算成了改正性维护——这是核心陷阱。改正性维护的前提是"交付后"发现的问题，交付前是开发阶段。

维护四种类型看"为什么改"：改环境→适应性，改bug→改正性（交付后！），加功能→完善性，防未来→预防性。关键陷阱：交付前发现的bug不算维护——那时系统还没交付，属于开发阶段。维护的时间起点是"交付"。

---



## 第 23 题

答案：A、B、D

黑盒测试方法：等价类划分、边界值分析、错误推测法。语句覆盖（C）是白盒方法。黑盒不看内部结构，只基于输入输出设计用例。

---

## 答案速查

| 题号 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 |
|----|---|---|---|---|---|---|---|---|---|----|----|----|
| 答案 | B | B | B | B | C | C | A | D | B | C  | A  | A  |

| 题号 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 | 21 | 22 | 23  |
|----|----|----|----|----|----|----|----|----|----|----|-----|
| 答案 | B  | B  | D  | B  | B  | A  | B  | B  | C  | A  | ABD |

